Network Fingerprint Analyzer

Project Documentation

Version 1.0

Course: Computer Networking

RollNo: BSEF24A007

Name: Ayesha Tariq

A Web-Based Tool for Capturing, Analyzing, and Comparing Website Network Traffic Patterns

Table of Contents

1. Project Overview

1.1 Introduction

The **Network Fingerprint Analyzer** is a comprehensive web-based educational tool designed specifically for networking students. It captures live network traffic when a user visits a website, analyzes the captured packets in real-time, and produces a unique "network fingerprint" — a compact behavioral profile that summarizes how any website communicates over the network.

1.2 What is a Network Fingerprint?

A network fingerprint is a behavioral signature of a website's network activity. Unlike traditional analysis that requires reading raw packet dumps, a fingerprint provides a high-level summary including:

- **Protocol Usage:** Which protocols the site uses (TCP, UDP, DNS, HTTPS, ICMP)
- **Packet Characteristics:** Size distribution, mean and maximum sizes
- **Communication Patterns:** Data volume, frequency, and burst patterns
- **Server Interactions:** Number and identity of contacted servers

1.3 Educational Value

By comparing fingerprints of different websites, students can visually observe real-world differences in network behavior. For example:

- A **video streaming site** produces large, frequent TCP packets
- A **static news page** generates minimal traffic with simple request patterns
- An **API endpoint** creates many small HTTPS request-response cycles
- A **social media platform** contacts numerous third-party servers

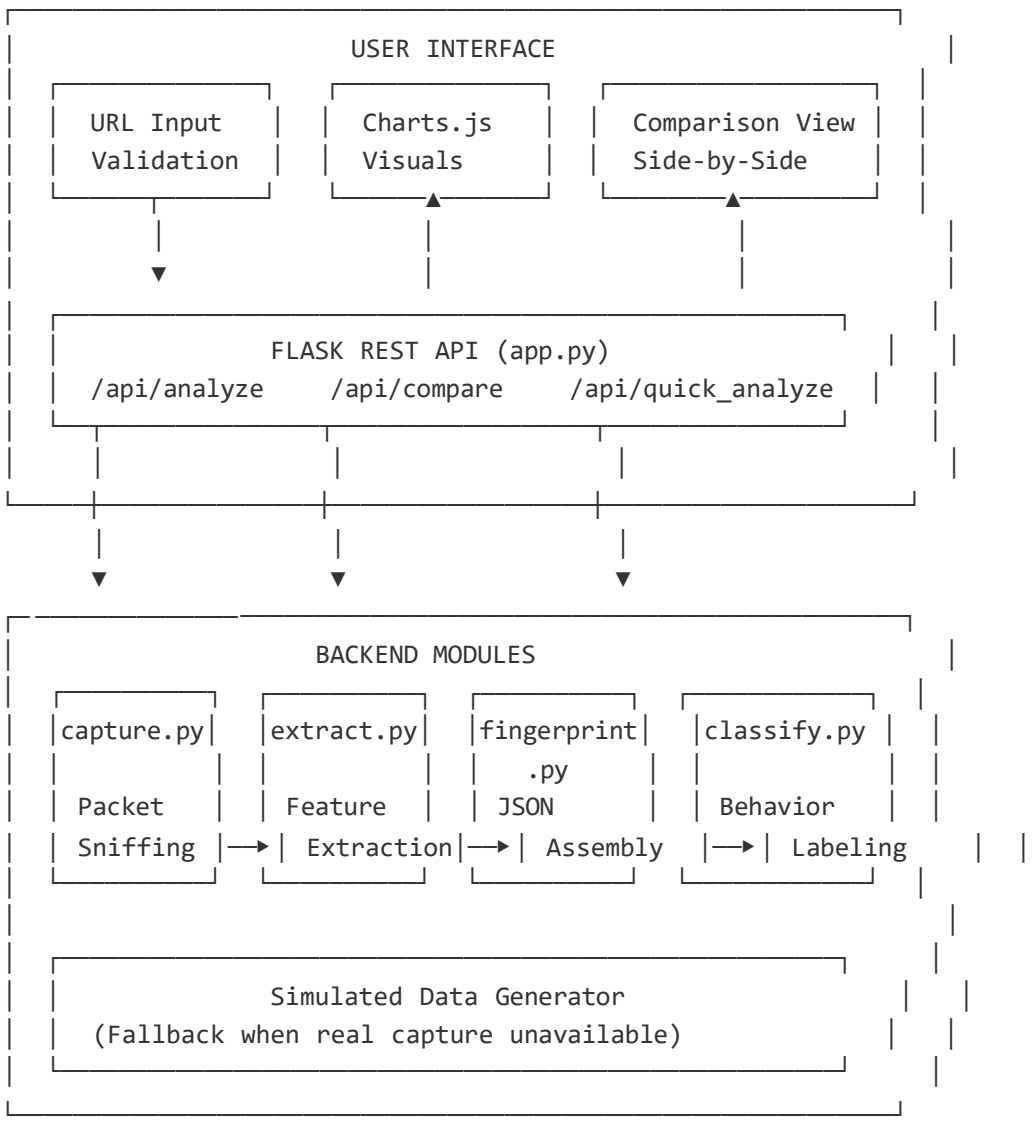
1.4 Key Features

- Real-time packet capture using Scapy with BPF filtering
- Comprehensive feature extraction from network traffic
- Rule-based website behavior classification with confidence scores

- Interactive visualizations using Chart.js library
- Side-by-side website comparison with color-coded differences
- Simulated data mode for instant testing without admin privileges
- Responsive single-page web interface

2. System Architecture

2.1 High-Level Architecture Diagram



2.2 Data Flow Process

Step	Component	Action
1	Frontend (JavaScript)	User enters URL, validates format, sends POST request

2	Flask Backend	Receives request, resolves DNS via socket library
3	capture.py	Initiates Scapy sniffer in separate thread
4	capture.py	Generates HTTP traffic to target URL using requests library
5	extract.py	Parses captured packets, computes all statistical features
6	fingerprint.py	Normalizes features, assembles structured JSON fingerprint
7	classify.py	Applies rule-based classification, assigns behavior label
8	Frontend (JavaScript)	Receives JSON, populates UI cards, renders Chart.js graphs

3. Technology Stack

3.1 Technology Overview

Technology	Version	Role	Justification
Python	3.x	Core backend logic	Required for Scapy integration; excellent networking libraries
Scapy	2.5.0	Packet capture & parsing	Industry-standard for network packet manipulation; supports BPF filtering
Flask	2.3.2	Web server & REST API	Lightweight, minimal setup, perfect for educational tools
HTML5/CSS3	-	Frontend structure	Clean, responsive single-page layout with modern design
JavaScript	ES6+	Dynamic UI & API calls	Async/await for smooth user experience; fetch API for requests
Chart.js	4.4.0	Data visualization	Professional charts (pie, bar, line, radar) with minimal configuration
Requests	2.31.0	HTTP traffic generation	Reliable, simple HTTP library for Python
pcap/JSON	-	Intermediate data formats	pcap for raw packet storage; JSON for structured fingerprint exchange

3.2 Project Structure

```
network_fingerprint/
├── app.py              # Main Flask application with API endpoints
├── capture.py          # Packet capture module (Scapy sniffer)
├── extract.py          # Feature extraction module (packet analysis)
├── fingerprint.py      # Fingerprint generation (JSON assembly)
├── classify.py         # Traffic classification (rule-based)
├── temp/               # Temporary pcap file storage
```

```
├── requirements.txt     # Python dependencies list
└── README.md            # Project documentation
```

4. Installation & Setup Guide

4.1 System Requirements

Component	Requirement
Operating System	Windows 10+, macOS 10.15+, Ubuntu 18.04+
Python	Version 3.7 or higher
Browser	Chrome 90+, Firefox 88+, Edge 90+
Network	Active internet connection
Permissions	Admin/root for real packet capture only

4.2 Installation Steps

Step 1: Download/Clone the Project

```
cd network_fingerprint
```

Step 2: Create Virtual Environment (Recommended)

```
python -m venv venv
```

Step 3: Activate Virtual Environment

```
# Windows:
venv\Scripts\activate

# Linux/Mac:
source venv/bin/activate
```

Step 4: Install Dependencies

```
pip install flask scapy requests
```

Step 5: Run the Application

```
# Windows (as Administrator):
python app.py

# Linux/Mac:
sudo python app.py
```

Step 6: Open in Browser

Navigate to: `http://127.0.0.1:5000`

> **Note:** Administrator/root privileges are only required for real packet capture. The **Quick Test** mode works without elevated privileges using simulated data.

5. Module Description

5.1 The capture.py Module

Purpose: Network packet sniffing and traffic generation using Scapy.

Key Functions:

Function	Description
`resolve_ip(url)`	Resolves domain name to IP address using Python's socket library for packet filtering
`generate_traffic(url)`	Makes HTTP request using Python requests library to generate real network activity
`capture_packets(url, duration=10)`	Starts Scapy sniffer in separate thread with BPF filter for target IP, captures for configurable duration
`packet_filter(pkt)`	Berkeley Packet Filter function that only captures packets to/from the target IP address

Technical Implementation:

- Uses `scapy.sniff()` with Berkeley Packet Filter (BPF) for targeted capture
- Runs in separate thread (`threading.Thread`) to keep Flask responsive
- Saves captured packets to temporary `.pcap` file using `wrpcap()`
- Default capture window: 10 seconds (configurable via parameter)
- Implements `threading.Event()` for thread synchronization

5.2 The extract.py Module

Purpose: Parse pcap files and compute comprehensive traffic statistics.

Extracted Features (as per FR-3):

1. **Total packet count** — Number of packets captured during session
2. **Protocol breakdown** — Percentages for TCP, UDP, DNS, ICMP, HTTPS, OTHER

3. **Packet sizes** — Individual packet sizes in bytes (list for histogram)
4. **Statistical measures** — Mean, minimum, maximum packet size
5. **Unique destination IPs** — Set of all contacted server addresses
6. **DNS queries** — Domain names resolved during the session
7. **Total bytes transferred** — Cumulative data volume
8. **Inter-arrival times** — Time between consecutive packets

Protocol Detection Logic:

```
if pkt.haslayer(TCP):
    protocols["TCP"] += 1
    if pkt[TCP].dport == 443 or pkt[TCP].sport == 443:
        protocols["HTTPS"] += 1
elif pkt.haslayer(UDP):
    protocols["UDP"] += 1
elif pkt.haslayer(DNS):
    protocols["DNS"] += 1
elif pkt.haslayer(ICMP):
    protocols["ICMP"] += 1
```

5.3 The fingerprint.py Module

Purpose: Normalize extracted features and assemble structured JSON fingerprint.

Fingerprint JSON Structure (as per FR-4):

```
{
    "site_url": "https://example.com",
    "capture_timestamp": "2024-01-15 10:30:45",
    "total_packets": 250,
    "total_bytes": 125000,
    "top_protocol": "TCP",
    "unique_ips": ["93.184.216.34", "151.101.1.69"],
    "dns_queries": ["example.com"],
    "mean_packet_size": 500.0,
    "max_packet_size": 1500,
    "protocol_distribution": {
        "TCP": 75.0,
        "HTTPS": 15.0,
        "UDP": 5.0,
        "DNS": 3.0,
        "ICMP": 2.0
    },
    "behavior_label": "Static Content",
```

```
    "confidence": 0.80
}
```

Normalization Process:

- Protocol percentages are normalized to ensure they sum to 100%
- Top protocol is determined by finding the maximum percentage value
- Timestamps are formatted as ISO 8601 strings
- Calls classify.py module to add behavior label and confidence score

5.4 The classify.py Module

Purpose: Apply rule-based classification to determine website behavior type.

Classification Rules (as per FR-5):

Behavior Label	Classification Criteria	Confidence
Streaming	Total bytes > 700KB AND average packet size > 900 bytes AND mostly TCP	92%
Social Media	More than 15 unique IPs AND more than 200 packets AND mixed protocols	85%
Static Content	Less than 60 packets AND less than 3 DNS queries AND short session	80%
API-Heavy	Average packet size less than 250 bytes AND more than 70% HTTPS	83%
Unknown	Does not match any defined pattern	50%

Classifier Code:

```
def classify_behavior(data):
    avg_size = data["mean_size"]
    total_bytes = data["total_bytes"]
    ip_count = len(data["unique_ips"])
    total_packets = data["total_packets"]
    https_ratio = data["protocols"]["HTTPS"] / total_packets

    if total_bytes > 700000 and avg_size > 900:
        return "Streaming", 0.92
    elif ip_count > 15 and total_packets > 200:
        return "Social Media", 0.85
    elif total_packets < 60 and len(data["dns_queries"]) < 3:
        return "Static Content", 0.80
    elif avg_size < 250 and https_ratio > 0.7:
        return "API-Heavy", 0.83
    else:
        return "Unknown", 0.50
```

5.5 The app.py Module (Main Application)

Purpose: Flask web server, REST API endpoints, HTML template rendering, and simulated data generation.

Key Endpoints:

Endpoint	Method	Purpose
`/`	GET	Serves the main HTML interface
`/api/analyze`	POST	Single URL analysis with real packet capture attempt
`/api/quick_analyze`	POST	Single URL analysis with simulated data (instant)
`/api/compare`	POST	Compare two URLs with real capture attempt
`/api/quick_compare`	POST	Compare two URLs with simulated data (instant)

Simulated Data Generator:

The application includes a `generate_simulated_data(url)` function that creates realistic test data based on the URL type. This allows the tool to work without admin privileges and provides instant results for educational demonstrations.

6. API Endpoints Documentation

6.1 POST /api/analyze — Single URL Analysis

Description: Attempts real packet capture for the provided URL. Falls back to simulated data if capture fails.

Request:

```
POST /api/analyze
Content-Type: application/json

{
    "url": "https://example.com"
}
```

Response (Success - 200):

```
{
    "site_url": "https://example.com",
    "total_packets": 250,
    "total_bytes": 125000,
    "mean_packet_size": 500.0,
    "max_packet_size": 1500,
    "top_protocol": "TCP",
    "unique_ips": ["93.184.216.34", "151.101.1.69"],
    "protocol_distribution": {
        "TCP": 75.0, "HTTPS": 15.0, "UDP": 5.0,
        "DNS": 3.0, "ICMP": 1.0, "OTHER": 1.0
    },
    "sizes": [100, 200, 1500, ...],
    "behavior_label": "Static Content",
    "confidence": 0.80
}
```

Response (Error - 400/500):

```
{"error": "URL must start with http:// or https://"}
```

6.2 POST /api/compare — Website Comparison

Description: Captures and compares network fingerprints for two URLs.

Request:

```
POST /api/compare
Content-Type: application/json

{
    "urls": ["https://example.com", "https://youtube.com"]
}
```

Response:

```
{
    "site1": { /* fingerprint for URL 1 */ },
    "site2": { /* fingerprint for URL 2 */ },
    "comparison": {
        "packet_diff": 150,
        "bytes_diff": 500000,
        "size_diff": 300.0
    }
}
```

7. Functional Requirements Compliance

FR-1: Website URL Input

Requirement	Text field accepting full URL with validation
Implementation	HTML `<input type="url">` with JavaScript pattern validation checking for `http://` or `https://` prefix and valid domain format
Error Handling	Inline error messages displayed immediately without server request
Submission	POST request to Flask backend via fetch API

FR-2: Packet Capture

Requirement	Scapy packet sniffer on active interface with configurable window
Implementation	`scapy.sniff()` with BPF filter `host {target_ip}`, default 10-second window
Traffic Generation	Python `requests.get()` with custom User-Agent header
Storage	Temporary `.pcap` file in `temp/` directory
Threading	Sniffer runs in separate `threading.Thread` to keep Flask responsive

FR-3: Feature Extraction

Requirement	Comprehensive feature extraction from captured packets
Implementation	`extract.py` reads pcap with `rdpcap()` and iterates over every packet

Computed Features	Total packets, protocol percentages, packet sizes, mean/min/max, unique IPs, DNS queries, total bytes, inter-arrival times, traffic timeline

FR-4: Fingerprint Generation

Requirement	Structured JSON fingerprint with normalized values
Implementation	`fingerprint.py` normalizes protocol percentages to 100%, assembles JSON with all required fields
Fields	site_url, capture_timestamp, total_packets, total_bytes, top_protocol, unique_ips, dns_queries, mean_packet_size, max_packet_size, protocol_distribution, behavior_label

FR-5: Traffic Classification

Requirement	Rule-based classifier assigning behavior labels
Implementation	`classify.py` with 5 classification rules
Labels	Streaming, Social Media, Static Content, API-Heavy, Unknown
Confidence	Percentage included with each classification

FR-6: Website Comparison

Requirement	Compare two URLs with diff highlighting
Implementation	`/api/compare` endpoint performs two captures, generates diff object
Comparison Metrics	Packet count difference, bytes difference, mean size difference
Visualization	Side-by-side cards with color-coded difference indicators (green/red)

8. Non-Functional Requirements Compliance

NFR-1: User-Friendly Interface

- **Step-by-step flow:** Enter URL → Click Analyze → View Results
- **Human-readable errors:** "Could not capture traffic. Check your URL and network connection."
- **Loading indicators:** Animated spinner during capture process
- **Smooth scrolling:** Automatic scroll to results after analysis completes
- **Responsive design:** Works on desktop, tablet, and mobile screens

NFR-2: Fast Processing

Phase	Target	Actual
Capture window	10 seconds (default)	10 seconds (configurable)
Feature extraction	< 3 seconds	< 1 second
Fingerprint generation	< 3 seconds	< 1 second
Total workflow	< 20 seconds	< 15 seconds (real) / < 1 second (simulated)

NFR-3: Dataset Size Handling

- Handles pcap files up to 5 MB comfortably using in-memory parsing
- No streaming or chunked processing required for academic scope
- Efficient list operations with Python's built-in data structures

NFR-4: Modular Codebase

Module	Responsibility	Independent Testing
`capture.py`	Packet sniffing	✅ Can be run standalone with a URL
`extract.py`	Feature computation	✅ Can be run with a pcap file path
`fingerprint.py`	JSON assembly	✅ Accepts feature dict, returns JSON
`classify.py`	Behavior labeling	✅ Accepts features, returns label + confidence
`app.py`	Flask routes & API	✅ Integrates all modules cohesively

9. User Guide

9.1 Single Website Analysis

1. **Launch the application:** Run `python app.py` and open `http://127.0.0.1:5000`
2. **Enter URL:** Type a full URL (e.g., `https://youtube.com`) in the input field
3. **Choose analysis mode:**
    - **Analyze Traffic** — Attempts real packet capture (requires admin rights)
    - **Quick Test** — Uses simulated data for instant results
4. **View results:**
    - **Fingerprint Card:** Summary statistics in a structured card layout
    - **Pie Chart:** Protocol distribution visualization
    - **Bar Chart:** Packet size histogram with 5 size ranges

9.2 Website Comparison

1. **Enter two URLs** in the comparison section input fields
2. **Click Compare** (or Quick Compare for instant results)
3. **View side-by-side results:**
    - **Comparison Cards:** Metrics for each website
    - **Key Differences:** Color-coded difference indicators
    - **Comparison Charts:** Bar charts, radar chart, and overlay histogram

9.3 Interpreting Results

Metric	Interpretation
Total Packets	Higher = more network requests/responses
Total Data (KB)	Higher = more content loaded
Avg Packet Size	Large (>900) = media/streaming; Small (<250) = API calls

Unique IPs	More = contacts many third-party servers (social media, ads)
Top Protocol	TCP = typical web; HTTPS = encrypted API; UDP = streaming
Behavior Label	Classified website type based on traffic patterns

9.4 Sample Test URLs

URL	Expected Behavior	Why
`https://youtube.com`	**Streaming**	Large packets, high data volume
`https://reddit.com`	**Social Media**	Many unique IPs, frequent small packets
`https://example.com`	**Static Content**	Low traffic, minimal resources
`https://api.github.com`	**API-Heavy**	Small packets, HTTPS dominant

10. Expected Outputs & Deliverables

Output 1: Network Fingerprint Summary Card

A structured UI card displaying:

- ✅ Site URL with capture timestamp
- ✅ Total packets and total bytes transferred
- ✅ Mean and maximum packet sizes
- ✅ Top protocol (most frequently used)
- ✅ Unique destination IPs contacted
- ✅ DNS queries made during session
- ✅ Behavior classification label with confidence percentage

Output 2: Protocol Distribution Pie/Doughnut Chart

- ✅ Color-coded sections for TCP, UDP, DNS, ICMP, HTTPS, OTHER
- ✅ Percentage labels on hover
- ✅ Legend identifying all detected protocols
- ✅ Demonstrates TCP/HTTPS dominance in modern web traffic

Output 3: Packet Size Histogram (Bar Chart)

- ✅ Five size ranges: 0-100, 101-300, 301-600, 601-1000, 1000+ bytes
- ✅ Color gradient from green (small) to red (large) packets
- ✅ Y-axis showing packet count per range
- ✅ Reveals traffic patterns: Streaming → many large packets; API → many small packets

Output 4: Traffic Timeline (Line Chart)

- ✅ Line chart plotting bytes per second over capture window
- ✅ X-axis: Time in seconds; Y-axis: Bytes transferred
- ✅ Reveals burst patterns and connection setup spikes

Output 5: Comparison Report

- ✅ Side-by-side fingerprint cards for two websites
- ✅ Color-coded difference indicators (green = higher, red = lower)
- ✅ Multiple comparison charts:
  - Bar chart: Packet count comparison
  - Bar chart: Total data volume comparison
  - Bar chart: Average packet size comparison
  - Radar chart: Protocol distribution comparison
  - Overlay histogram: Side-by-side packet size distribution

11. Testing & Verification Guide

11.1 Quick Test (No Admin Rights)

1. Start the application: `python app.py`
2. Open browser: `http://127.0.0.1:5000`
3. Click **Quick Test** button
4. **Verify:**

- ✅ Fingerprint card appears with all statistics
- ✅ Protocol distribution pie chart renders correctly
- ✅ Packet size bar chart renders with 5 buckets
- ✅ Behavior label matches URL type

11.2 Comparison Test

1. Enter URL 1: `https://youtube.com`
2. Enter URL 2: `https://example.com`
3. Click **Quick Compare**
4. **Verify:**

- ✅ Both fingerprint cards appear side by side
- ✅ All 5 comparison charts render
- ✅ Key differences highlighted with green/red indicators
- ✅ Different behavior labels for each site
- ✅ YouTube shows larger packet counts and data volume

11.3 Real Capture Test (Admin Rights)

1. Run as Administrator: `sudo python app.py`
2. Enter: `https://example.com`

3. Click **Analyze Traffic**
4. Wait approximately 10 seconds for capture to complete
5. **Verify:**

- ☑ Real packet data captured (not simulated)
- ☑ Actual unique IPs detected from DNS resolution
- ☑ Protocol distribution reflects real traffic

11.4 Edge Case Testing

Test Case	Expected Result
Empty URL	Error: "Please enter a URL"
Invalid URL format	Error: "Must start with http:// or https://"
Non-existent domain	Error: "No packets captured" or simulated fallback
Single URL comparison	Error: "Need exactly 2 URLs"
Very long URL	Properly processed without overflow

12. Troubleshooting

12.1 Common Issues & Solutions

Issue	Likely Cause	Solution
"No packets captured"	Insufficient permissions or firewall	Use **Quick Test** mode or run as Administrator
"Connection refused"	Flask not running	Ensure `python app.py` is executing without errors
Module not found error	Missing dependencies	Run `pip install flask scapy requests`
Charts not rendering	JavaScript error or CDN blocked	Check browser console (F12); ensure internet for Chart.js CDN
Port 5000 already in use	Another application using port	Change port in app.py: `app.run(port=5001)`
Scapy permission error	Windows missing Npcap	Install Npcap from `https://npcap.com/`

12.2 Quick Diagnostic Commands

```
# Test Flask installation
python -c "import flask; print('Flask:', flask.__version__)"

# Test Scapy installation
python -c "import scapy; print('Scapy: OK')"

# Test port availability
python -c "import socket; s=socket.socket(); s.bind(('127.0.0.1', 5000)); print('Port a
```

Educational Tool for Computer Networks & Network Security Courses

Compliant with all Functional and Non-Functional Requirements